

Apuntes del 27/08/26

Dayron Josué Porras Pérez - 2023124693

“Parientes” de los compiladores

Hay programas relacionados con compiladores.

Ahora mucho software tiene interconexión con compiladores.

Cómo los sistemas operativos y los lenguajes de alto nivel han sido muy exitosos, actualmente el desarrollo de software se asocia a su uso.

Podemos usar lo de este curso en otras áreas (más que todo sus técnicas, así que si, este curso parece ser útil)



Jaja soy un
compilador

Ensambladores

“Queremos algo de alto nivel en bajo nivel”, así que los ensambladores nacen en los 1950s.

Como ya los hemos visto y somos del TEC, somos monstruos peludos en esto y podemos frontear cualquier ensamblador.

Aunque no parezca, todavía viven.

Tienen una relación de 1 a 1 con el lenguaje máquina y muy eficiente con sintaxis simple.

Ensamblador vs TEC



En ensamblador...

Hay campos fijos:
Etiqueta, código de
operación, argumentos.

Usan técnicas de
compiladores.

Típicamente dan 2
pasadas.

Más pasadas → más fácil
el problema → pero más
lento.

Los algoritmos de
compiladores se usan
porque suelen ser
eficientes, aunque antes
eran re lentos.

Storytime: cómo antes se
duraba como 15 minutos
por compilación, no se
hacía nada y nomas se
podía decir 'Be quiet man,
estoy compilando'

Como funciona la primer pasada en ensamblador?



El location counter (LC, que funciona como el PC en compiladores) analiza el código.



Si ve en el código y argumentos puede determinar 'mirá, ahí hay un shift a la izquierda'.



De alguna manera el LC sabe por cuál byte va, entonces sabe al final cuantos Bytes necesita en todo el programa.

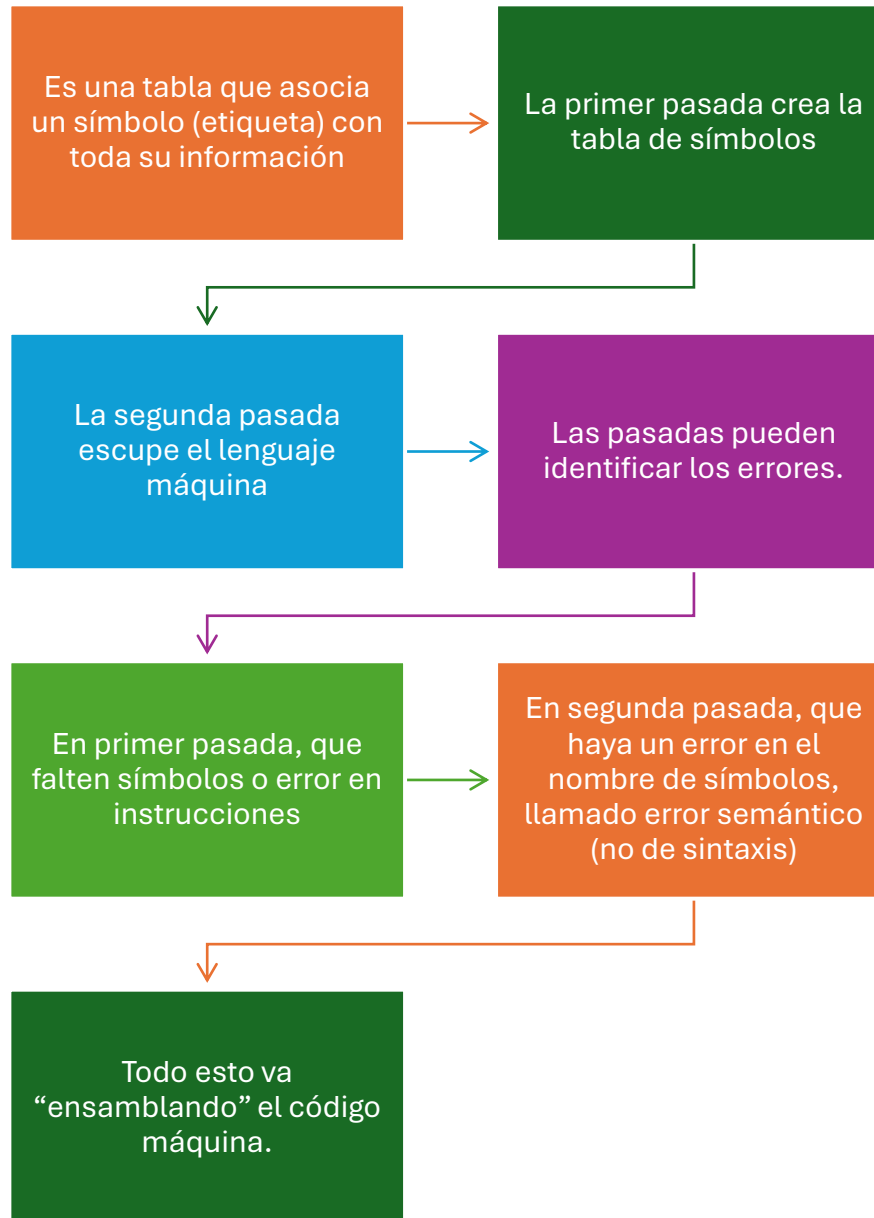


Si sabe dónde está una etiqueta, el JMP puede ir ahí...



Entonces cada que encuentra un símbolo lo agrega a la tabla de símbolos.

Tabla de símbolos



Símbolo	Dirección
ACA	778
....	
....	
FIN	1289
....	
....	
....	

Macros

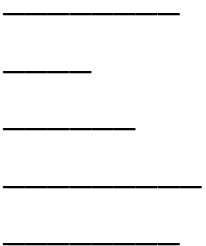
Se volvieron ambiciosos con los ensambladores.

Entonces nacieron las macros (contenidas en micro-ensambladores)

La macro es una hilera, la cual nunca tiene una llamada, sino un reemplazo de código.

En la tabla de símbolos se guarda cómo ‘Rutina’ con todo el body asociado, así como su longitud.

Rutina Def Macro



End Macro



Símbolo	Dirección
ACA	778
....	
Rutina	BodyAsociado, 16 bytes
....	
FIN	1289
....	
....	

Cómo puedo guardarlas?

Puede hacerse una pasada 0, para poder ver donde se definen estas macros.

O hacerlo en primer pasada, pero limita que las macros siempre estén al inicio del programa.

En la rutina aparecen los Bytes, entonces se los suma al LC para guardar la dirección en la que se encuentra.

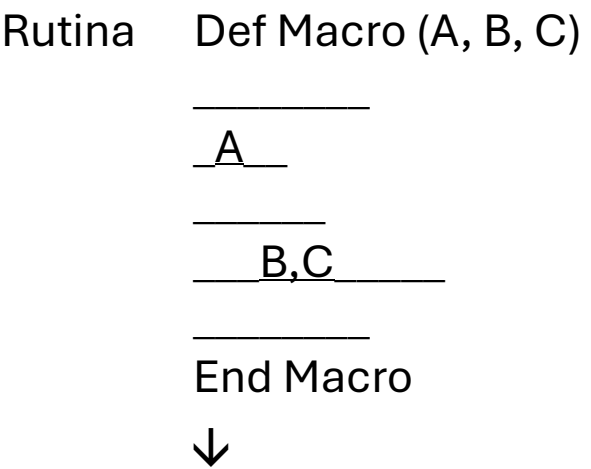
Y cómo nos gusta complicarnos...

Aparecen las macros con argumentos.

Ahora cada argumento es reemplazado dentro de la macro, y en la tabla de símbolos se le asocia una lista enlazadas de argumentos.

Eso sí... no se equivoque qué le formateo el disco.

Con el sombrero de programador esto es una maravilla, pero siendo chico compiladores es una tortura... al final alguno va a sufrir.



Símbolo	Dirección
ACA	778
....	
Rutina	BodyAsociado, 16 bytes, [A, B, C]
....	
FIN	1289
....	
....	

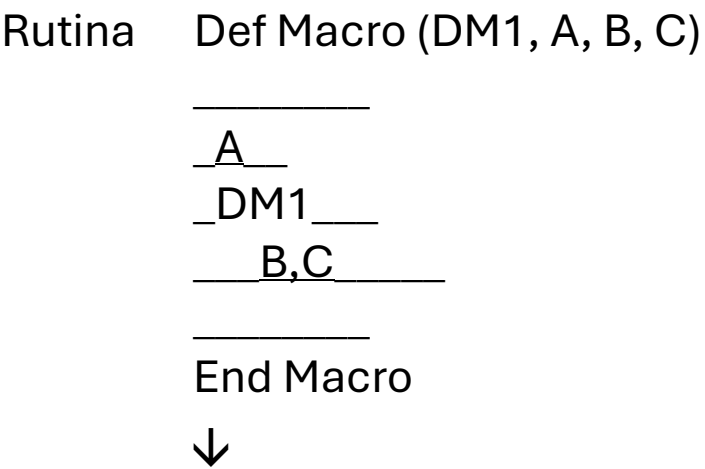
Sigamos complicando!

Si se quiere una macro con una macro adentro...

Da paso al nacimiento de las macros anidadas!

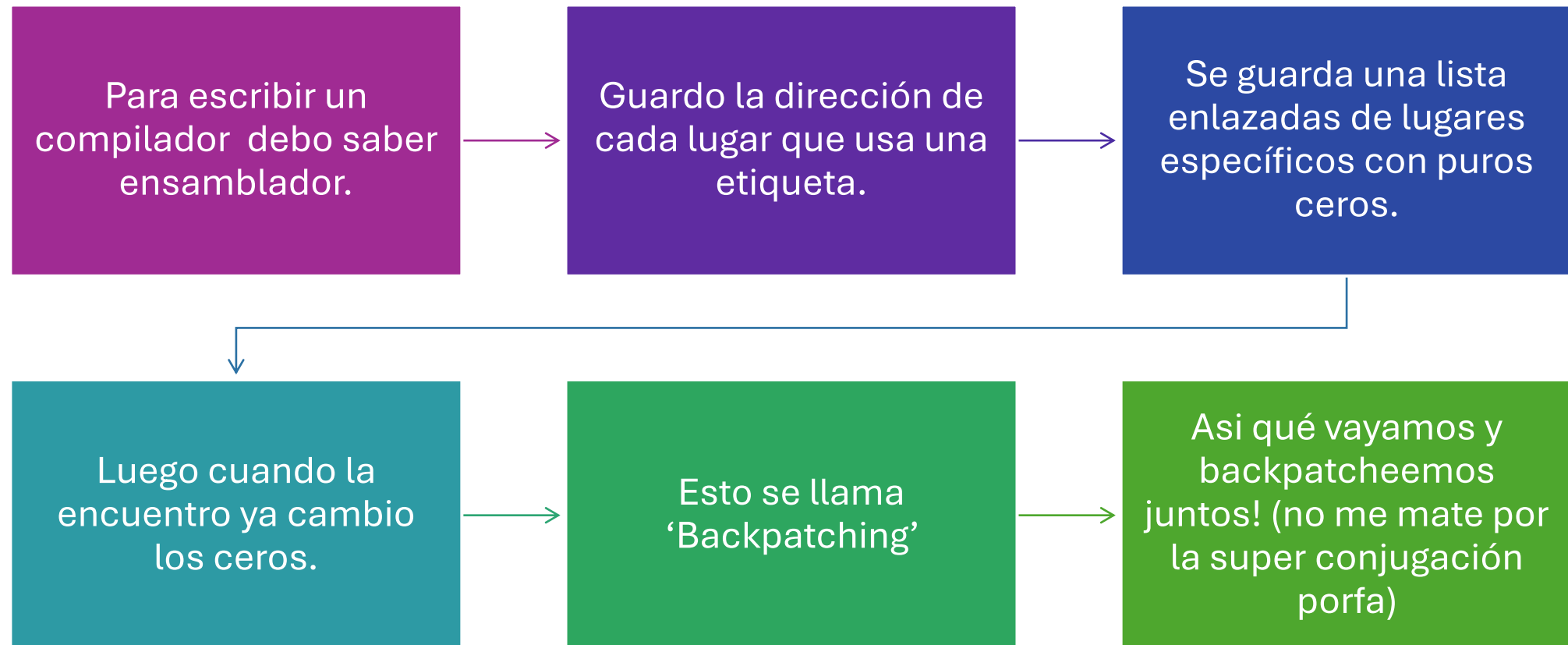
Primero debe existir la rutina de más adentro para que pueda existir la otra... o hagamos más pasadas.

Y si se preguntan... sí podrían haber macros recursivas con un ensamblador condicional.



Símbolo	Dirección
ACA	778
....	
Rutina	BodyAsociado, 16 bytes, [DM1, A, B, C]
....	
FIN	1289
....	
....	

Hacer esto en 1 pasada...



Desensamblador

- Cambia de código máquina a ensamblador.
- Se arregla con más pasadas.
- Sigue el proceso inverso a un ensamblador.
- Recuerda a un simulador de una arquitectura: debe conocer formatos binarios y reconoce códigos de operación y formatos de direccionamiento.
- Así como existe esto, también existen los descompiladores.
- Genera etiquetas y nombres de variables arbitrarias, porque nunca se van a entender los nombres ni las macros al 100%.

```
0000] > pd
0x00000000  90          nop
0x00000001  90          nop
0x00000002  6800009c00  push 0x9c0000 ; 0x00
0x00000007  e8c7ace37b  call 0x7be3acd3
               0x7be3acd3(unk)
0x0000000c  bb04009c00  mov ebx, 0x9c0004
0x00000011  8903       mov [ebx], eax
0x00000013  e81903f47b  call 0x7bf40331
               0x7bf40331( )
0x00000018  bb08009c00  mov ebx, 0x9c0008
0x0000001d  8903       mov [ebx], eax
0x0000001f  bb00009c00  mov ebx, 0x9c0000
0x00000024  c60300     mov byte [ebx], 0x0
-> 0x00000027  68e8030000  push 0x3e8 ; 0x000000
0x0000002c  e81124e37b  call 0x7be32442
               0x7be32442(unk)
<= 0x00000031  ebf4       jmp 0x100000027
0x00000033  90          nop
0x00000034  ff         invalid
0x00000035  ff         invalid
0x00000036  ff         invalid
0x00000037  ff         invalid
```

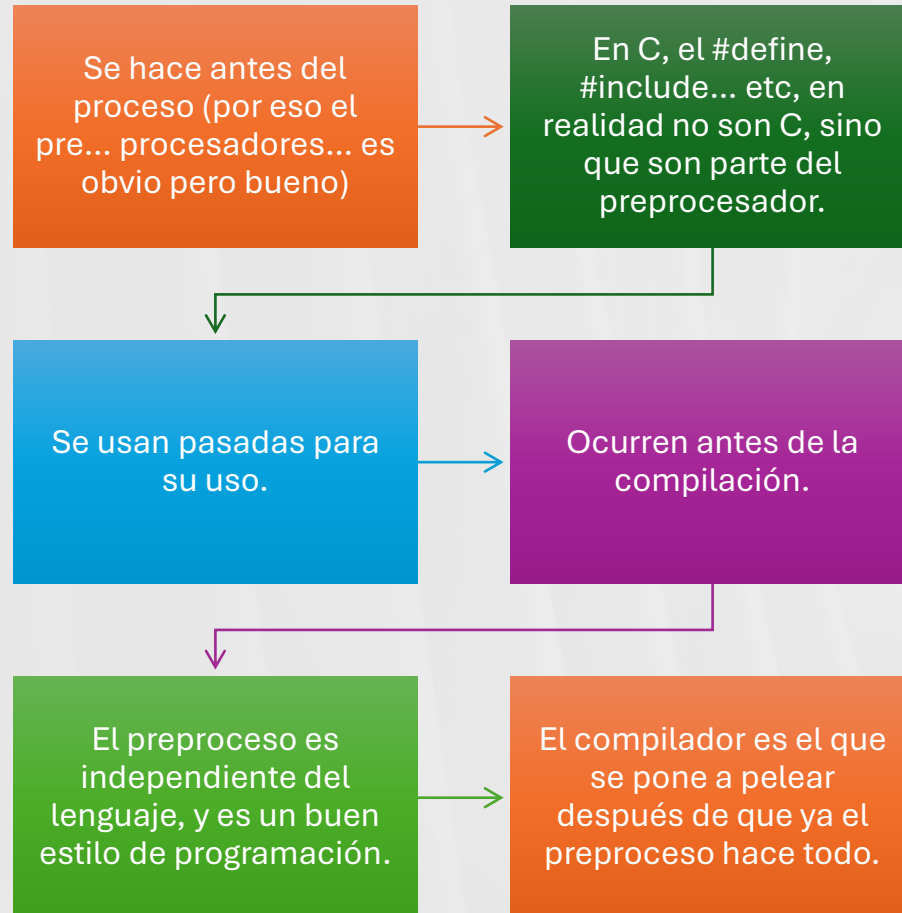
Este es un código en desensamblador y cómo ven... las variables y etiquetas son puro dígito raro.

Intérpretes

- Combinan traducción con ejecución.
- Los intérpretes van línea por línea.
- Usan muchas técnicas de compiladores.
- Son independientes de la arquitectura.
- Es un Run-Time environment. Un software que corre siempre que se ejecuta.
- Son lentos en ejecución, ya que todo se repite cada vez que se ejecuta.



Preprocesadores



Compilador vs Código preprocesado



Imagen reciclada, okay?

Funciones de preprocesamiento

Tablita con todas las que
encontré...

Preprocesador	Qué hace
#include	Para insertar un encabezado específico de un archivo.
#define	Se utiliza principalmente como reemplazo de una macro de preprocesador.
#ifdef	Si la macro está definida, devuelve verdadero. Y si no, entonces falso.
#endif	Se utiliza principalmente para cerrar la directiva del preprocesador.
#undef	Para desdefinir un encabezado estándar o definido por el usuario.
#pragma	Para habilitar y deshabilitar ciertas funciones.
#ifndef	Devuelve verdadero cuando macro no está definido.
#if	Comprueba si la condición es verdadera en el momento de la compilación.
#else	Comprueba la siguiente condición si #if resulta ser falso en tiempo de compilación.
#elif	Se utiliza como una combinación de #else y #if.
#error	Se utiliza principalmente para imprimir errores en stderr.

Muchas gracias, esto fue todo



Procesador



PreProcesador.....??



El ensamblador y el lenguaje máquina en su relación 1 a 1